Towards Efficient Parameter Utilization in Transformer with Cross-Layer Sharing of Experts

Dian Jiao[*], **Jiaxin Duan**[*], **Shuai Zhao, Jian Wang, Jiabing Leng, Yiran Zhang, Feng Huang**[†]

China Electronics Cloud Technology Co., Ltd.

Abstract

While Transformer-based architectures have revolutionized Large Language Models (LLMs), existing designs suffer from substantial inter-layer parameter redundancy, where dimensional transformations in Feed-Forward Network (FFN) layers or sparse experts are independently re-learned across network depths. In this paper, we investigate this parameter redundancy problem and propose Self-MoE, a novel architecture that implements cross-layer expert reusability. Unlike traditional MoEs that rely on layer-isolated experts, Self-MoE equips each Transformer layer with a small set of independent experts while providing simultaneous access to a large, centralized shared expert pool. This hybrid structure enables precise control over both token-level parameter activation and computational budgets (FLOPs) by scaling the shared pool and tuning the activation density. Our architectural innovation allows Self-MoE to transcend traditional parameter capacity limits through dynamic compute allocation. Extensive experiments demonstrate three key advantages: (1) *Superior Efficiency*: Compared to dense baselines, Self-MoE achieves significantly lower perplexity while activating only 55% of the parameters. (2) *Scalable Performance*: With a fixed parameter scale, increasing expert activation counts yields continuous performance gains, effectively utilizing additional FLOPs. (3) *Architectural Convergence*: Under fixed FLOPs, expanding the shared expert pool reduces expert repetition and drives performance toward the upper bound of standard MoEs. Ultimately, Self-MoE establishes a flexible Pareto frontier between computational expenditure and model capacity, offering a high-efficiency alternative to standard MoEs for FLOP-constrained environments.

1 Introduction

The proliferation of Transformer-based architectures has served as a pivotal advancement in artificial intelligence, providing the representational capacity and parallelization necessary to catalyze the success of Large Language Models (LLMs). However, as these models scale toward trillion-parameter regimes, the traditional **Dense Transformer** encounters diminishing marginal returns in efficiency [6, 10]. Recent empirical inquiries into weight pruning and model compression [11, 8] suggest that dense models suffer from significant **activation redundancy**, wherein a substantial portion of parameters contributes negligibly to the final output for any given input [5]. This phenomenon indicates that standard Feed-Forward Networks (FFNs) contain latent capacity that remains underutilized during inference, leading to a suboptimal ratio of information density to computational expenditure.

To mitigate this inefficiency, the **Mixture-of-Experts (MoE)** paradigm [13, 7] has emerged as the *de facto* standard for scaling LLMs. By substituting monolithic FFN layers with a set of sparse experts

[*]Equal contribution.

[†]Corresponding author: huangfeng01@cestc.cn

Preprint.

and selectively activating a subset (Top-K) per token, MoE architectures effectively decouple total model capacity from active computational cost (FLOPs). Nevertheless, while MoE alleviates the computational burden, its conventional implementation - defined by **layer-isolated experts** - introduces a secondary inefficiency termed **parameter redundancy**. In standard MoE designs, experts are confined to specific layers, preventing the network from exploiting functional recurrence across different stages of the hidden state transformation.

Our research is motivated by a growing body of work encompassing advanced MoE architectures [1, 2, 9, 4], which suggests that the rigid isolation of parameters in sparse architectures is often unnecessary. These studies reveal that strategic parameter reuse and dynamic allocation can significantly enhance architectural efficiency without incurring a substantial sacrifice in modeling performance. Building upon this trajectory, *we hypothesize that the physical barriers between layers in traditional MoEs lead to an underutilization of expert capacity*, as similar functional transformations are often redundantly learned at multiple depths. In Figure 1, we investigate the potential for cross-layer expert sharing. Empirical validation through zero-shot substitution demonstrates that replacing deep-layer experts with functionally equivalent shallow counterparts - identified via CKA (see Appendix A for details) - incurs moderate perplexity degradation, significantly outperforming random replacement and confirming that experts are largely interchangeable across depths.

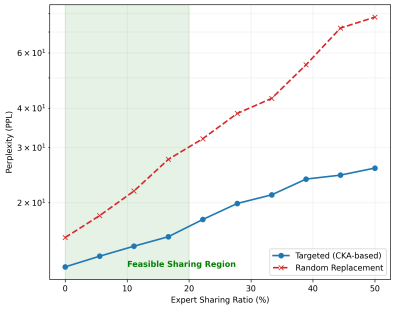


Figure 1: PPL of MoE models with random expert replacement and deliberate expert replacement based on CKA analysis.

Building upon these insights, we propose **Self-MoE**, a novel Transformer architecture that implements a "super-sharing" mechanism to bridge the gap between sparse capacity and parameter efficiency. Self-MoE restructures the traditional expert distribution into a **dual-tier hierarchy** that distinguishes between localized specialization and global functional reuse. Instead of the conventional layer-isolated approach, we introduce a **dual-path execution strategy**: a **Fixed Path** comprising a small set of **Independent Experts** that are unconditionally activated at each layer to capture depth-specific features (e.g., hierarchical syntax or task-specific projections), and a **Dynamic Path** that provides all layers with concurrent access to a centralized **Shared Expert Pool**. By employing a layer-specific router to select K experts from this global repository, the architecture allows tokens to invoke universal semantic operators regardless of their position in the network stack. This design principle ensures that the model maintains the hierarchical integrity of the Transformer while simultaneously benefiting from the massive parameter capacity afforded by a shared global pool.

We validate the efficacy of Self-MoE through a series of rigorous experiments ranging from 0.6B to 12B parameter scales. Our results demonstrate three primary technical advantages. First, we establish **Superior Parameter Density**; at a fixed computational and parameter budget, Self-MoE achieves a significant reduction in perplexity compared to dense baselines, requiring only **55%** of the parameters to be active per forward pass. Second, we demonstrate **Scalable Compute Performance**, where model performance scales monotonically with the expert activation count (K), allowing Self-MoE to transcend the capacity limitations inherent in traditional fixed-parameter designs. Finally, we characterize the **Flexible Pareto Frontier** of the architecture, showing that in FLOP-constrained environments, Self-MoE provides a high-efficiency alternative that converges toward the performance upper bound of standard MoEs as the shared pool is scaled. Through these contributions, Self-MoE establishes a new equilibrium between computational expenditure and model capacity, providing a scalable pathway for deploying memory-efficient LLMs.

2 Methodology

The architecture of **Self-MoE** is centered on the hypothesis that optimal model capacity is achieved not through independent scaling of layers, but through a hierarchical organization of parameters. By bifurcating the Feed-Forward Network (FFN) into a dense layer-specific path and a sparse global path, Self-MoE addresses inter-layer redundancy while maintaining the specialized capacity required for deep hierarchical processing.

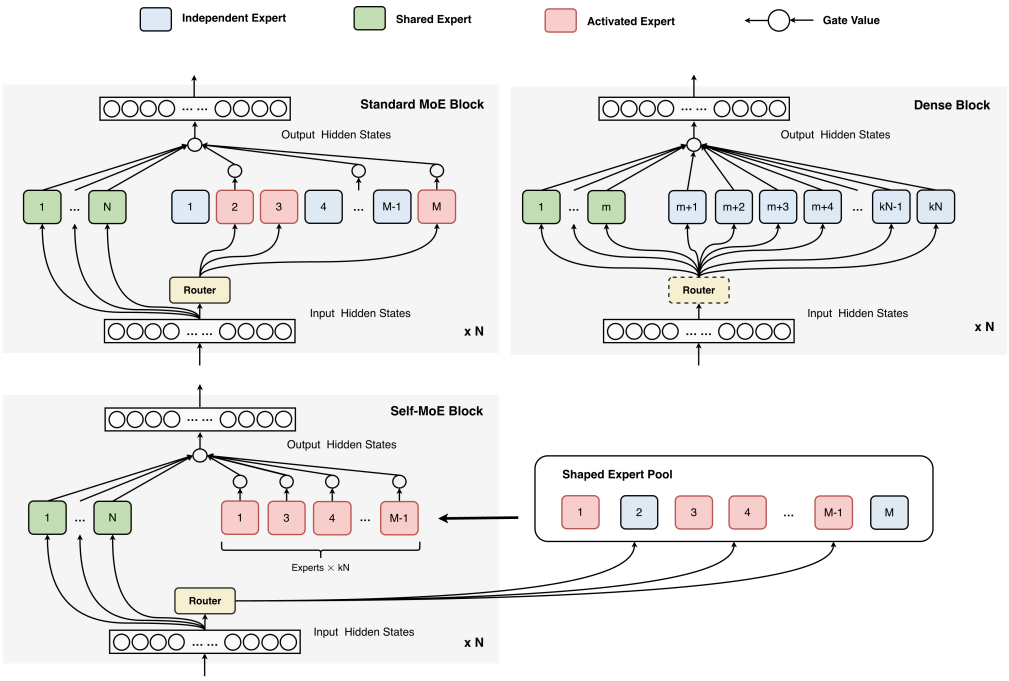


Figure 2: Comparison of Transformer architectures: standard MoE (upper left), dense (upper right), and Self-MoE (lower). The dense Transformer has a virtual router marked in a dashed box.

2.1 Unified Topology of Transformers

To provide a rigorous foundation for Self-MoE, we formalize the Transformer Feed-Forward Network (FFN) through a generalized Mixture-of-Experts (MoE) framework. A standard **Dense Transformer** represents a constrained instance of this topology where the model employs a single, "always-on" expert ($N = 1, K = 1$) at every layer. In contrast, a **Sparse MoE Transformer** scales the parameter count by utilizing a larger pool of experts per layer ($N > 1$) but restricts computation by activating only a small subset ($K \ll N$).

Self-MoE generalizes these paradigms into a hybrid, dual-tier topology. It posits that a fixed portion of the transformation parameters should be "anchored" to specific depths to capture localized features, while a significantly larger, centralized pool should be shared across all layers to handle recurrent semantic patterns. By breaking the physical isolation between layers, Self-MoE enables a single expert to participate in the computation of multiple stages of the network, thereby maximizing parameter utility and facilitating a more efficient equilibrium between model capacity and computational expenditure.

2.2 Self-MoE Architecture

As illustrated in Figure 3, Self-MoE restructures each Transformer layer $l \in \{1, \ldots, L\}$ into a dual-path execution model: a **Fixed Path** consisting of layer-specific Independent Experts and a **Dynamic Path** drawing from a global Shared Expert Pool. The total expert set available to the network is partitioned as follows:

$$\mathcal{E}_{total} = \left(\bigcup_{l=1}^{L} \mathcal{E}_{indep}^{(l)} \right) \cup \mathcal{E}_{shared}$$

where $\mathcal{E}_{indep}^{(l)} = \{e_1^{(l)}, \ldots, e_N^{(l)}\}$ contains N independent experts private to layer l, and $\mathcal{E}_{shared} = \{s_1, \ldots, s_M\}$ is a large, centralized repository of M experts accessible by every layer in the Transformer stack. In this configuration, the execution logic for each layer follows two distinct principles:

- **Routing-free Fixed Path:** Each layer l activates its own N **Independent Experts** ($\mathcal{E}_{indep}^{(l)}$) unconditionally for every token. This ensures a baseline of depth-specific transformation capacity without the overhead of routing, effectively maintaining the reliability of dense architectures.
- **Sparse Dynamic Path:** The routing mechanism in Self-MoE is exclusively restricted to the **Shared Expert Pool**. For each token at layer l, a layer-specific router selects and activates kN experts from $\mathcal{E}_{shared}$. Crucially, because the shared pool is universal, a specific shared expert s_j may be activated simultaneously across multiple layers for the same or different tokens, allowing for extreme longitudinal parameter reuse.

Consequently, the total computation of a single layer is composed of N independent experts and kN routed shared experts, resulting in a total of $(k+1)N$ activated experts per layer. This hybrid structure allows Self-MoE to simulate the capacity of an ultra-wide MoE while maintaining a lean physical parameter footprint.

2.3 Hybrid Activation and Routing Logic

The execution of a Self-MoE layer is defined by the simultaneous processing of these two paths. Given a token representation $\mathbf{h}_l \in \mathbb{R}^d$, the routing mechanism manages only the dynamic path (the shared pool).

1. **Shared Path Routing:** The layer-specific router R_l computes scores $\mathbf{s}_l$ for the M shared experts:

$$\mathbf{s}_l = W_{r,l}\mathbf{h}_l$$

   where $W_{r,l} \in \mathbb{R}^{M \times d}$. We then apply a Top-$K$ gating function to select the most relevant shared experts:

$$\mathbf{g}_l = \text{Softmax}(\text{TopK}(\mathbf{s}_l, K))$$

2. **Output Aggregation:** The final output of the layer, $\mathbf{o}_l$, is the sum of the dense independent experts and the sparsely activated shared experts:

$$\mathbf{o}_l = \underbrace{\sum_{j=1}^{N_{indep}} \text{Expert}_{e,j}^{(l)}(\mathbf{h}_l)}_{\text{Fixed Path}} + \underbrace{\sum_{i \in \text{TopK}} g_{l,i} \cdot \text{Expert}_{s,i}(\mathbf{h}_l)}_{\text{Dynamic Path}}$$

This design ensures that every token benefits from N_{indep} specialized parameters tailored to the current layer's depth, while simultaneously drawing from K experts in the global pool. By adjusting K and M, the architecture can precisely tune the ratio of compute-to-parameters, effectively breaking the linear relationship between model depth and parameter count.

2.4 Layer-wise Load Balancing for Shared Experts

To prevent the shared pool from suffering from expert collapse - where a few experts dominate the routing across all layers - we implement a layer-wise auxiliary load-balancing loss $\mathcal{L}_{aux}^{(l)}$. This loss is calculated specifically for the shared expert pool at each layer to ensure diverse utilization.

For a batch of T tokens, the balancing loss is defined as:

$$\mathcal{L}_{aux}^{(l)} = M \sum_{i=1}^{M} f_{l,i} \cdot P_{l,i}$$

Where $f_{l,i}$ is the fraction of tokens in the batch that selected shared expert i at layer l, and $P_{l,i}$ is the average routing probability assigned to that expert. The total objective function is:

$$\mathcal{L}_{total} = \mathcal{L}_{LM} + \alpha \sum_{l=1}^{L} \mathcal{L}_{aux}^{(l)}$$

where α is the balancing coefficient. This ensures that the shared pool remains a robust, multi-functional resource throughout the training process.

Table 1: Detailed model hyperparameters (Qwen3-MoE Backbone). *: Top-K activated experts include independent experts of each layer.

Model Type	Total Params	Activated Params	Layers (L)	d_{ffn}	d_{exp}	N_{indep}	Shared Pool (M)	Top-K^*
Dense	0.6B / 1.7B / 4B / 8B	Same	16 - 36	3,072 - 12,288	-	-	-	-
Self-MoE 0.6B	0.6B	0.6B / 0.9B / 1.7B	28	3,072	768	1	84	4 / 10 / 21
Self-MoE 1.7B	1.7B	1.7B / 4B	28	6,144	1,536	1	84	4 / 13
Self-MoE 4B	4B	4B / 8B	36	9,728	2,432	1	108	4 / 10
Self-MoE 8B	8B	4B	36	9,728	2,432	1	324	4
Self-MoE 8B	8B	8B	36	12,288	3,072	1	324	4
Self-MoE 12B	12B	4B	36	9,728	2,432	1	540	4

3 Implementation

3.1 Model Configurations and Backbone

To evaluate the efficacy of **Self-MoE**, we implement three architectural paradigms: **Dense Transformers**, **Standard MoE Transformers**, and our proposed **Self-MoE**. All models are developed using the **Qwen3-MoE**[3] architecture as the foundational backbone, inheriting its optimized implementations of Grouped-Query Attention (GQA), SwiGLU activation functions, and Rotary Positional Embeddings (RoPE).

The configurations across various scales are detailed in Table 1, where d_{ffn} and d_{ffn} are the dimensions of the feed-forward layer and expert, respectively. To reconstruct the standard Qwen3-MoE architecture as our Self-MoE, we split a feed-forward layer into four experts at each layer. For example, Self-MoE 0.6B has four experts per layer with a dimension of 768. Additionally, we employ a **Total (Activated)** notation to delineate the relationship between the physical parameter footprint in high-bandwidth memory (HBM) and the actual computation performed per token.

In the Self-MoE variants, the **Fixed Path** consists of a single independent expert per layer ($N_{indep} = 1$). In contrast, the **Dynamic Path** utilizes a shared expert pool (M) where only K experts are activated per token. This structure allows the 8B Self-MoE variant, with all parameters activated, to achieve competitive performance while activating only approximately **55%** of its parameters.

3.2 Training Protocol

All models are pre-trained on a massive integrated corpus comprising **WuDao** [12] and **DCLM** [3], ensuring a robust mixture of multi-lingual knowledge and high-quality web-scale data. To maintain a rigorous and controlled comparison, we unify the training hyperparameters across all experimental groups, as specified in Table 2.

Table 2: Unified Training Hyperparameters

Hyperparameter	**Value**
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95, \epsilon =$ 1e-8)
Learning Rate	3e-4
Weight Decay	0.01
Batch Size	512 (effective token count $\approx$ 1M)
Warmup Steps	1,000
Total Steps	12,500
LR Schedule	Cosine Annealing to $0.1 \times \text{LR}_{\max}$
Context Length	2048

The implementation utilizes a customized version of the Megatron library[4] to efficiently handle the routing logic of the shared expert pool across 8 Nvidia H200 GPUs. For MoE-based models, the auxiliary load-balancing coefficient α is fixed at 0.01 to ensure expert utilization without destabilizing the language modeling objective.

[3] `https://github.com/huggingface/transformers/tree/main/src/transformers/models/qwen3_moe`

[4] `https://github.com/NVIDIA/Megatron-LM`

4 Experiments

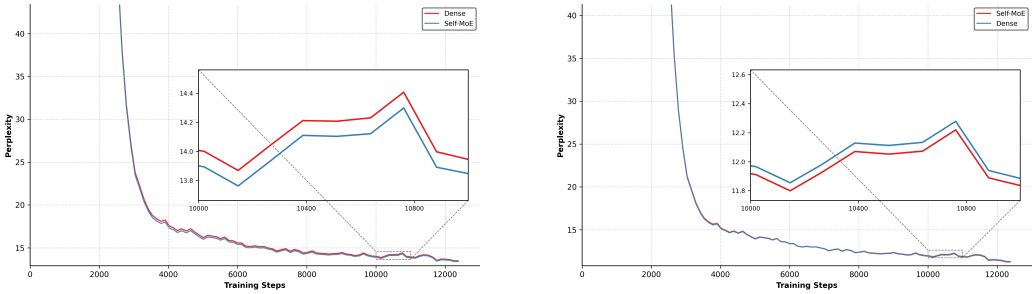
We conduct three distinct groups of experiments to empirically validate the architectural superiority of Self-MoE, focusing on efficiency, scalability, and structural convergence.

To quantify the architectural efficiency of the super-sharing mechanism, we define the **Expert Utilization Ratio (EUR)** (ρ), which represents the proportion of unique shared experts activated across the entire network relative to the total expert pool:

$$\rho = \frac{\left|\bigcup_{l=1}^{L} \mathcal{S}_l\right|}{\min(M, \delta)}$$

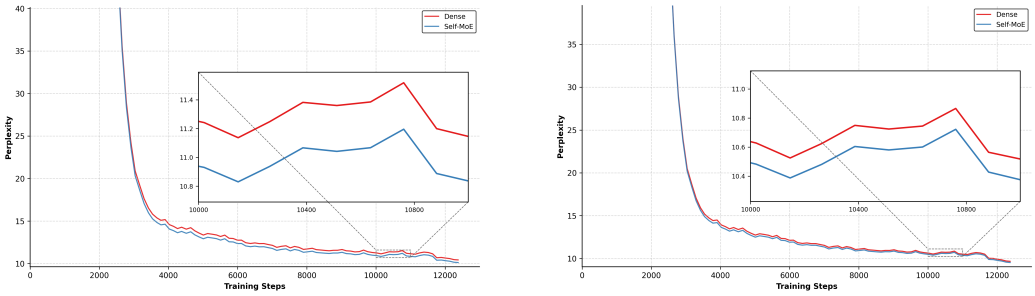
where L is the number of layers, N is the number of independent experts per layer, M is the total size of the shared expert pool, and $\mathcal{S}_l$ denotes the subset of kN shared experts activated at layer l. Notably, δ accumulates the activated experts across all layers, which may exceed M as k increases.

4.1 Efficiency Gains and Parameter Expansion



(a) PPL of 0.6B Models.

(b) PPL of 1.7B Models.



(c) PPL of 4B Models.

(d) PPL of 8B Models.

Figure 3: Comparison between Self-MoE and Dense models.

In our first study, we compare Self-MoE against Dense Transformers while keeping the total parameter count and FLOPs aligned. As shown in Figure 3, Self-MoE consistently outperforms Dense baselines across all scales. Firstly, we observe a "Parameter Expansion" effect: for example, the Self-MoE model with an 8B total parameter footprint, but only **55%** of those parameters activated per token, achieves a significantly lower perplexity (PPL) than a Dense model of the same total size. This suggests that the hybrid expert pool allows the model to leverage a wider knowledge base than its dense counterpart without increasing the computational burden per forward pass. Additionally, scaling the size of Self-MoE introduces significantly more performance gain than increasing the activations, suggesting that the breadth of the shared expert

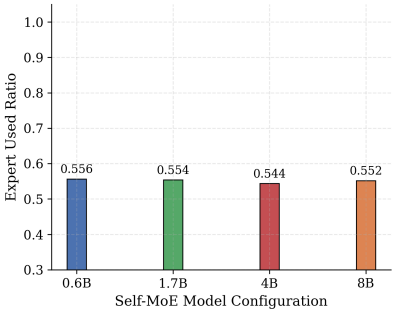
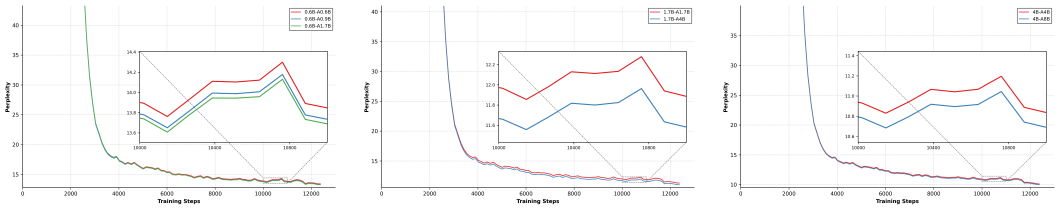


Figure 4: The EUR of Self-MoE increases with scaling model size.

pool is more critical for performance than the density of per-token routing. This highlights the importance of global parameter availability in enhancing model capacity.

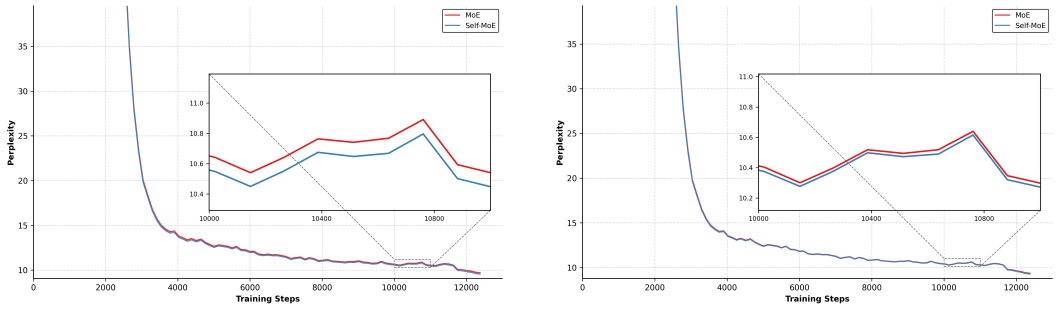


(a) PPL of 0.6B-AxB Models. (b) PPL of 1.7B-AxB Models. (c) PPL of 4B-AxB Models.

Figure 5: Comparison among Self-MoE models with varying activations.

The second group of experiments investigates the relationship between the expert activation count (K) and model performance while holding the total parameter scale fixed. In Figure 5, we increase K in the 0.6B, 1.7B, and 4B Self-MoE variants so that the model can activate more than its actual parameters per token. We then observe a monotonic improvement in performance. Unlike standard architectures where capacity is strictly tied to parameter count, Self-MoE demonstrates that **increasing token-level compute allocation (FLOPs)** - by activating more experts from the shared pool - can effectively bypass the traditional "Parameter-Compute bottleneck." This result confirms that the shared expert pool acts as a scalable resource that the model can query more intensively to yield continuous performance gains.

4.2 Architectural Convergence and Sharing Dynamics



(a) PPL of 8B-A4B Models. (b) PPL of 12B-A4B Models.

Figure 6: Comparison between Self-MoE and Sparse MoE models with equal activations.

Finally, we analyze the convergence behavior of Self-MoE relative to Standard MoEs under fixed FLOP constraints. We increase the model size (as well as the size of the shared expert pool (M)) and measure the repetition rate of expert activation. Results in Figure 7 indicate that as M expands, the **expert utilization ratio** across layers significantly increases to around 1.0. According to Figure 6, the performance of Self-MoE asymptotically approaches that of a standard, layer-isolated MoE. This suggests that Self-MoE functions as a generalized architecture: at small M, it provides extreme parameter efficiency through high reuse; as $M \to \infty$, it preserves the modeling power of a standard MoE. This flexibility allows Self-MoE to define a Pareto frontier that is particularly advantageous in FLOP-constrained or memory-constrained deployment scenarios.

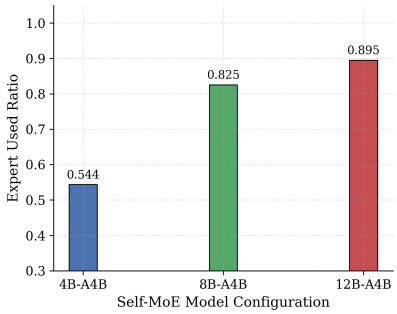


Figure 7: Expert used ratio of Self-MoE models with 4B activations.

5 Conclusion

This paper presents **Self-MoE**, a novel Transformer architecture designed to address inter-layer parameter redundancy through a dual-tier expert hierarchy. By decoupling depth-localized independent experts from a centralized shared pool, Self-MoE facilitates cross-layer functional reuse while maintaining hierarchical specialization. Extensive evaluations demonstrate that Self-MoE achieves superior perplexity compared to dense baselines while activating fewer parameters. Furthermore, the architecture establishes a flexible Pareto frontier between computational expenditure and model capacity, allowing performance to scale monotonically with the number of expert activations. In the future, we will explore efficient expert parallel mechanisms for Self-MoE and pull it to resource-limited deployment scenarios.

Moving forward, we aim to explore:

- **Generalization**. Scaling Self-MoE to ultra-large regimes, such as 30B and 80B parameter models, to examine the robustness of super-sharing at the extreme frontier.
- **Advanced Sharing Strategy**. The impact of non-uniform allocation by varying the number of independent experts at different layers.
- **Depth-aware routing**, which involves interventions to modulate expert reuse patterns across network stages, further refining the balance between localized specialization and global knowledge sharing.

References

[1] Anonymous. Moe++: Accelerating mixture-of-experts methods with efficient all-to-all communication. *arXiv preprint arXiv:2410.07347*, 2024.

[2] Anonymous. Seqtopk: Sequence-level top-k selection for non-autoregressive moe. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[3] Apple, University of Washington, Tel Aviv University, et al. Datacomp-lm: In search of the next generation of training sets for language models. *arXiv preprint arXiv:2406.11794*, 2024.

[4] Zewen Chi et al. X-moe: Sparse models with sparse experts. *arXiv preprint arXiv:2204.09179*, 2022.

[5] OpenReview Contributors. Sparseswaps: Tractable llm pruning mask refinement at scale. In *The Thirteenth International Conference on Learning Representations*, 2025.

[6] Damai Dai, Chengqi Deng, Shuo Zhao, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.

[7] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts, 2024.

[8] Dmitry Lepikhin, Hyoukseok Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

[9] Bill Yuchao Lin et al. Dynamoe: Dynamically mitigating negative transfer in multi-task learning. *arXiv preprint arXiv:2408.00000*, 2024.

[10] Niklas Muennighoff et al. Olmoe: Open mixture-of-experts language models. *arXiv preprint arXiv:2501.00662*, 2025.

[11] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[12] Sha Yuan, Hanyu Zhao, Zhengxiao Du, Ning Ding, Xiao Liu, Yukuo Cen, Xu Huang, et al. Wudaocorpora: A super large-scale chinese corpora for pre-training language models. *AI Open*, 2:65–68, 2021.

[13] Longfei Yun, Yonghao Zhuang, Yao Fu, Eric P. Xing, and Hao Zhang. Toward inference-optimal mixture-of-expert large language models. *ArXiv*, abs/2404.02852, 2024.

A Detailed Implementation of the Pilot Study

We conducted our analysis using a pre-trained Qwen3-MoE-4B model, which features 36 Transformer layers and 4 experts per layer ($N = 4$, Top-$K = 1$). The study was performed on a held-out validation set of 50,000 tokens sampled from the DCLM corpus to ensure statistical significance in activation patterns.

Representational Similarity Analysis. To quantify the functional overlap between experts without being misled by the permutation invariance of neural weights, we employed Centered Kernel Alignment (CKA). Unlike Cosine similarity, CKA is invariant to orthogonal transformations and isotropic scaling, making it the standard for comparing internal representations.

For two expert weight matrices $W_i^{(l)}$ and $W_j^{(l')}$ from layers l and l', we computed the linear CKA score as follows:

$$CKA(K, L) = \frac{HSIC(K, L)}{\sqrt{HSIC(K, K)HSIC(L, L)}}$$

where $K = XX^T$ and $L = YY^T$ are the Gram matrices of the activations produced by the respective experts, and $HSIC$ is the Hilbert-Schmidt Independence Criterion. A high CKA score (> 0.85) between a shallow-layer expert and a deep-layer expert served as our primary indicator of longitudinal redundancy.

Zero-Shot Expert Substitution Protocol. To verify the interchangeability of experts, we implemented a greedy substitution algorithm:

1. **Selection:** For each expert in the target "Deep" layers (layers 24-32), we identified a "Source" expert from the "Shallow" layers (layers 1-8) that maximized the CKA score.
2. **Replacement:** We performed a direct parameter swap, replacing the weights of the Deep expert with the weights of its Shallow counterpart.
3. **Inference:** We measured the change in model perplexity on the validation set without any gradient updates or fine-tuning.
4. **Baseline:** As a control, we performed random substitutions of equal magnitude.

The moderate increase in perplexity (under 0.15 within acceptable region) during targeted substitution, compared to a sharp divergence in the random baseline, empirically justifies the feasibility of the shared expert pool proposed in Self-MoE.